

SENTINEL

Setting up the machine you build on

ECE 476 Engineering System Software • Fall 2026 • University of Victoria

What this covers

SENTINEL is built and run on Linux. This paper sets up a Linux system on the computer you own, and then installs the tools all four labs use. Do this once. It serves Lab 1 to Lab 4.

Read the section for your own computer, then read **Every machine**, which applies to all of them.

One warning before you start, because it costs marks. If you use Windows, your work must live inside the Linux file system and not on the Windows drive. Section 3 states where, and why the acceptance test fails if you get it wrong.

Which processor you need

Either of the two common ones works. Nothing in the course prefers one.

Your computer	What you install	<code>uname -m</code> says
Mac, Apple silicon (M1 to M4)	Ubuntu for aarch64 , in a virtual machine	aarch64
Mac, Intel	Ubuntu for x86_64 , in a virtual machine	x86_64
Windows	Ubuntu through WSL2	x86_64
Linux	nothing. You already have it	either

1 Mac with Apple silicon

An Apple silicon Mac has an **aarch64** processor, so it runs **aarch64** Linux at full speed in a virtual machine. Use UTM, which is free.

1. Install UTM from <https://mac.getutm.app/>.
2. Download the Ubuntu Server **ARM64** image from <https://ubuntu.com/download/server/arm>. Take the current long term support release.
3. In UTM choose **Create a New Virtual Machine**, then **Virtualize**, then **Linux**. Select the image you downloaded.
4. Give the machine 4 GB of memory, 2 processors and 32 GB of disk. Less than 2 GB is not enough to build and run the labs together.
5. Start the machine and complete the Ubuntu installer. Choose to install OpenSSH when the installer offers it, because it lets you work from the Mac terminal.

Choose **Virtualize** and not **Emulate**. Virtualize runs the Linux code directly on your processor. Emulate translates every instruction, and the labs become too slow to measure.

2 Mac with an Intel processor

The steps are the same as Section 1, with two differences. Download the Ubuntu Server **AMD64** image instead of ARM64, and `uname -m` then reports `x86_64`.

VirtualBox from <https://www.virtualbox.org/> also works on an Intel Mac.

3 Windows

Windows runs Linux through WSL2, the Windows Subsystem for Linux. You do not need a virtual machine.

1. Open PowerShell as Administrator and run:

```
wsl --install -d Ubuntu
```

2. Restart the computer when it asks.

3. Start **Ubuntu** from the Start menu. It asks for a user name and a password the first time. That password is for Linux, and it is not your Windows password.

4. Check that you have version 2, which the labs need:

```
wsl -l -v
```

The **VERSION** column must say 2. If it says 1, convert it:

```
wsl --set-version Ubuntu 2
```

3.1 Where your work must live

Keep your repository in the Linux home directory. Do not keep it on the Windows drive.

Inside Ubuntu, your Linux home directory is `~`, which is `/home/<your name>`. The Windows drives appear under `/mnt/c`, `/mnt/d` and so on. Both are reachable, and only one of them works.

Path	Use it?
<code>~/sentinel-lab</code>	Yes. A real Linux file system
<code>/mnt/c/Users/you/sentinel-lab</code>	No. A Windows drive

Why it matters. Lab 1 creates the mission log with permissions `0600`, meaning the owner can read and write it and nobody else can. It is a surveillance record. The acceptance test checks that the permissions are exactly `0600`, and it checks another file for exactly `0644`.

A Windows drive does not store Unix permissions. Your program asks for `0600`, the Windows drive cannot record that, and the file reports something else. The acceptance test then reports a failure in code that is correct, and nothing in the message points at the file system. Students lose hours to this.

Work in `~` and the problem does not exist.

To open your Linux files from Windows, type `\\wsl$` in File Explorer, or run `explorer.exe .` from the Ubuntu terminal.

4 Linux

You need nothing from this section. Go to **Every machine** below.

Every machine

Everything after this point runs inside Linux, whichever route brought you there.

Check the processor

```
uname -m
```

It prints `aarch64` or `x86_64`. Both are supported. The course supplies its programs built for both, so the answer only tells you which ones you receive.

Install the tools

```
sudo apt update
```

```
sudo apt install build-essential python3 valgrind git zip manpages-dev
```

Tool	What the labs use it for
<code>gcc</code> , <code>make</code>	compile and link your library, your tests and your demonstration program
<code>python3</code>	serve the operations centre display to your browser
<code>valgrind</code>	find memory faults and leaks in your code
<code>git</code>	receive each lab when the course releases it
<code>zip</code>	build the file you submit
<code>manpages-dev</code>	the manual pages for the system calls, which the prelab asks you to read

Check that the compiler answers:

```
gcc --version
```

```
make --version
```

Check that a build works

Once the course has given you the address of the course repository, and you have made your own private copy of it, clone that copy and build it. The Lab 1 manual, Section 4, states how to make the copy.

```
cd ~
```

```
git clone <your repository>
```

```
cd sentinel-lab
```

```
make
```

Before you have written any code, that prints a line like this, and it is correct:

```
src/libsyscall: no Makefile yet
```

`make` builds the given code, copies the given programs into `build/`, and then builds each lab that has a `Makefile`. You have not written yours yet, so it says so and stops. If you reach that line, the machine is ready.

Reach the display in your browser

The labs publish an operations centre display. You start it with:

```
make watch
```

Then open `http://localhost:8080/` in a browser. How you reach it depends on how you installed Linux.

Where you run Linux	How to open the display
WSL2 on Windows	Use any Windows browser. WSL2 forwards <code>localhost</code> for you
A virtual machine	Use a browser inside the virtual machine. To use the browser on the Mac instead, forward the port over SSH with <code>ssh -L 8080:localhost:8080 <user>@<vm></code>
Linux	Use any browser on the machine

If you work over SSH

Forward the port when you connect, and `http://localhost:8080/` then works on the computer in front of you:

```
ssh -L 8080:localhost:8080 <user>@<host>
```

If something does not work

What you see	What to do
<code>gcc: command not found</code>	<code>build-essential</code> is not installed. Run the install command again
The acceptance test reports wrong file permissions	On Windows, your repository is on a Windows drive. Move it to <code>~</code> . See Section 3
A manual page is missing	Install <code>manpages-dev</code>
The browser cannot reach <code>localhost:8080</code>	<code>make watch</code> must still be running in its own terminal. In a virtual machine, forward the port or use the browser inside the machine
The build or the labs run very slowly	In UTM, the machine was created with Emulate . Create it again with Virtualize